PRÁCTICA 7.2

SQL en Piloto Automático

CICLO FORMATIVO

TÉCNICO EN DAW

Noel David Núñez Martínez

César Valverde Pardo

CURSO 2025-2026

Módulo: Base de Datos

Murcia (Alhama) IES Miguel Hernández

ÍNDICE

Introducción	3
Procedimiento 1 - Ejercicio 1	3
Procedimiento 2 - Ejercicio 2	3
Procedimiento 3 - Ejercicio 3	4
Procedimiento 4 - Ejercicio 4	5
Procedimiento 5 - Ejercicio 5	5
Procedimiento 6 - Ejercicio 6	6
Función 1 - Ejercicio 7	7
Función 2 - Ejercicio 8	8
Función 3 - Ejercicio 9	8
Función 4 - Ejercicio 10	9
Función 5 - Ejercicio 11	10
Función 6 - Ejercicio 12	10
Cursor 1 - Ejercicio 13	11
Cursor 2 - Ejercicio 14	13
Cursor 3 - Ejercicio 15	14
Conclusiones personales	**15**

Introducción

En esta práctica trabajaremos el desarrollo de consultas avanzadas en SQL implementando procedimientos almacenados y funciones junto al manejo de errores. Además, analizaremos el uso de cursores.

Procedimiento 1 - Ejercicio 1

Crear un procedimiento almacenado llamado “GetProductCountByProductLine” que permita consultar el número de productos por líneas de producto (productLine). Tenga en cuenta el control de errores

Sentencia SQL:

```
SQL
CREATE DEFINER=`root`@`localhost` PROCEDURE
`GetProductCountByProductLine`(
       IN productline VARCHAR(50),
    OUT numProductLine INT
)
BEGIN
       SELECT  COUNT(*)
    INTO numProductLine
    FROM products
       WHERE productline = productLine;

    IF numProductLine = 0 THEN
             SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'ProductLine not found';
        END IF;
END
```

Explicación: Creamos el procedimiento, le declaramos un IN que será lo que queremos buscar y un OUT que será el dato que devolverá, seleccionamos un count(*) para que cuente toda las líneas y las almacenamos en numProductLine, y si el numProductLine es 0, entonces sale el control de errores, que mostrará “ProductLine not found”.

Procedimiento 2 - Ejercicio 2

Crear un procedimiento denominado “IncProductPercentage” que recibe 2 parámetros, el código de un producto y el valor de incremento (%) y que actualice el MSRP con el porcentaje:

Sentencia SQL:

```SQL
CREATE DEFINER=`root`@`localhost` PROCEDURE `IncProductPercentage`(
       IN p_productCode VARCHAR(15),
    IN incrementValue DECIMAL(10,2)

)
BEGIN
       UPDATE products
    SET MSRP = MSRP + (MSRP*incrementValue)/100
    WHERE productCode = p_productCode;
END
```

Explicación: Para este, le dAMOS los 2 parámetros que necesitaremos y para proseguir, usamos un UPDATE de la tabla de products, y con el SET le indicamos qué fila tiene que actualizar y la operacion para esto (MSRP + el valor añadido)

Procedimiento 3 - Ejercicio 3

Crear un procedimiento almacenado llamado “product pricing” que acepta tres parámetros: pl para almacenar el precio más bajo del producto, ph para almacenar el precio más alto del producto y para almacenar el precio promedio del producto (de todos los precios de productos en la tabla “order details”).
Sentencia SQL:

```SQL
CREATE DEFINER=`root`@`localhost` PROCEDURE `product pricing`(

       OUT pl DECIMAL(10,2),
    OUT ph DECIMAL(10,2),
    OUT pAvg DECIMAL(10,2)
)
BEGIN
       SELECT AVG(priceEach)
    INTO pAvg
    FROM orderDetails;

    SELECT MAX(priceEach)
             INTO ph
       FROM orderdetails;

    SELECT MIN(priceEach)
             INTO pl
       FROM orderdetails;
END
```

Explicación: Para esta hemos tenido que usar el AVG, el MIN y el MAX, para cada uno, seleccionamos la media del priceEach, el máximo del priceEach y el mínimo, y entonces lo metemos a cada una de sus variables.

Procedimiento 4 - Ejercicio 4

Crear un procedimiento almacenado denominado “Ordertotal” que dado un número de pedido calcule el importe total del mismo. Tenga en cuenta el control de errores
Sentencia SQL:

```
SQL
CREATE DEFINER=`root`@`localhost` PROCEDURE `Ordertotal`(
       IN numPedido INT(11),
    OUT total INT
)
BEGIN
       SELECT SUM(quantityOrdered*priceEach)
    INTO total
    FROM orderdetails
    WHERE numPedido = orderNumber
    GROUP BY orderNumber;

    IF total = 0 THEN
              SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Número del pedido no encontrado';
        END IF;
END
```

Explicación:Le pasamos como parámetro de entrada el número de pedido y conseguimos el total del pedido conseguido mediante una multiplicación de SUM (`quantityOrdered*priceEach)` , y luego hacer el control de errores.

Procedimiento 5 - Ejercicio 5

Crear un procedimiento almacenado llamado “NumPaymentsbyCountry” que dado un país indique el número de pagos total realizado por todos los clientes que residan en dicho país. Tenga en cuenta el control de errores
Sentencia SQL:

```
SQL
CREATE DEFINER=`root`@`localhost` PROCEDURE `NumPaymentsbyCountry`(
       IN c_country VARCHAR(50),
    OUT pagosRealizados INT
)
```

```
BEGIN
       SELECT COUNT(*)
    INTO pagosRealizados
    FROM payments
    INNER JOIN customers USING (customerNumber)
    WHERE country = c_country;
       IF pagosRealizados = 0 THEN
              SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Número del pedido no encontrado';
        END IF
END
```

Explicación: Le pasamos 2 parámetros, el in del country que queremos comprobar y el out para saber los datos de los pagos realizados, seleccionamos un count y el conteo lo metemos en pagosRealizados, unimos las tablas y lo restringimos para que la country sea igual a la country que le mandamos.

Procedimiento 6 - Ejercicio 6

Crear un procedimiento denominado “PendingPayments”. Descripción: Deberá calcular los pagos pendientes dado un código de cliente. Para saber si un cliente tiene algún pago pendiente deberemos calcular cuál es el importe total de todos los pedidos y de los pagos que ha realizado. Si el importe total de los pedidos es mayor que el de los pagos entonces ese cliente tiene pagos pendientes. Se mostrará “Pagos_Pendientes” o “Sin_Pagos_Pendientes”
Sentencia SQL:

```
SQL
CREATE DEFINER=`root`@`localhost` PROCEDURE `PendingPayments`(
       IN codcliente INT,
    OUT pagospendientes VARCHAR(50)
)
BEGIN
       DECLARE totalpagos DECIMAL default 0;
    DECLARE totalpedidos DECIMAL default 0;

       SELECT SUM(quantityOrdered*priceEach)
    INTO totalpedidos
    FROM orderdetails
    INNER JOIN orders USING(orderNumber)
    INNER JOIN customers USING(customerNumber)
    WHERE codcliente = customerNumber;

    SELECT SUM(amount)
    INTO totalpagos
```

```
    FROM payments
    INNER JOIN customers USING(customerNumber)
    WHERE codcliente = customerNumber;

    IF totalpedidos > totalpagos  THEN
    SET pagospendientes = 'Sin Pagos Pendientes';
    ELSE SET pagospendientes = 'Con Pagos Pendientes';
    END IF;

END
```

Explicación: Hacemos un select sum de amount que son los totalpagos y el otro para totalpedidos, para luego comprobar que si el totalpedidos es mayor que totalpagos, diga que no tiene pagos, y si es al revés, sí tiene pagos pendientes

Función 1 - Ejercicio 7

Crear una función llamada “CustomerRisk” que toma como parámetro el código de cliente y devuelve un mensaje que indica el riesgo asociado al cliente dependiendo del valor de creditlimit: Creditlimit > 50000 ‘LOW’; Creditlimit <= 50000 and >= 10000 ‘MEDIUM’; Creditlimit < 10000 and > 0 ‘HIGH’; Creditlimit = 0 ‘RED_ALERT’
Sentencia SQL:

```SQL
CREATE DEFINER=`root`@`localhost` FUNCTION `CustomerRisk`(codcliente INT
) RETURNS varchar(50) CHARSET utf8mb4 COLLATE utf8mb4_general_ci
BEGIN
       DECLARE riesgo VARCHAR(50);
    DECLARE credito DECIMAL(10,2);

    SELECT creditLimit INTO credito
    FROM customers
    WHERE codcliente = customerNumber;

    IF credito > 50000 THEN
             SET riesgo = 'LOW';
    ELSEIF (credito <=50000 AND credito >= 10000) THEN
             SET riesgo = 'MEDIUM';
       ELSEIF (credito < 10000 AND credito > 0) THEN
             SET riesgo = 'HIGH';
       ELSE
             SET riesgo = 'RED ALERT';
       END IF;
```

```
RETURN riesgo;
END
```

Explicación: Declaramos la variable riesgo y crédito (Uno para luego retornarlo y el otro para condicionar en el IF. Seleccionamos el creditoLimite y lo metemos dentro de la variable de crédito y una vez hecho eso, con el IF empezamos a condicionarlo para "LOW/MEDIUM/HIGH/RED_ALERT"

Función 2 - Ejercicio 8

Crear una función llamada "calcProfit" que toma dos parámetros de entrada: el precio de compra (buyPrice) y el precio sugerido por el proveedor (MSRP). Calcula la ganancia restando el precio de compra del precio sugerido y luego devuelve el valor a la expresión de llamada
Sentencia SQL:

```
SQL
CREATE DEFINER=`root`@`localhost` FUNCTION `calcProfit`(buyPricep
DECIMAL(10,2),
    MSRPp DECIMAL(10,2)
) RETURNS decimal(10,2)
BEGIN
       DECLARE gananciaRetornada DECIMAL(10,2);

    SET gananciaRetornada = (MSRPp-buyPricep)

RETURN gananciaRetornada;
END
```

Explicación: Creamos 2 parámetros, buyPricep y MSRPp, luego declaramos la variable de la ganancia
Hacemos un select de los parámetros que le damos, los restamos y los metemos en gananciaRetornada
Lo condicionamos para que coincida el MSRP y buyPrice con el nuestro y retornamos la ganancia

Función 3 - Ejercicio 9

Crear una función denominada "Count_Orders_Year" que toma de entrada un año en concreto y devuelve el número de pedidos que se enviaron en ese año (shippedDate).
Sentencia SQL:

```
SQL
CREATE DEFINER=`root`@`localhost` FUNCTION `Count_Orders_Year`(anyo INT
) RETURNS int(11)
```

```
BEGIN
       DECLARE totalPedidos INT;

    SELECT COUNT(orderNumber) INTO totalPedidos
    FROM orders
    WHERE YEAR(shippedDate) = anyo AND status = 'Shipped';

RETURN totalPedidos;
END
```

Explicación: Declaramos el año cómo un INT ya que no le damos la fecha entera
Luego hacemos un count de los pedidos y los ingresamos en pedidos, pero lo condicionamos para que el shippedDate coincida con el año y que el estado sea “Shipped” (Enviado)

Función 4 - Ejercicio 10

Crear función “Rupture_Stock” que informe sobre ruptura de stock de un artículo. La función debe recibir dos parámetros, el código del artículo y el nivel de ruptura de stock por debajo del cual debe informar si está ese artículo por debajo del mínimo. La función devolverá Stock-Rupture si el stock del artículo está por debajo del nivel proporcionado y Stock-Enough si no está por debajo
Sentencia SQL:

```
SQL
CREATE DEFINER=`root`@`localhost` FUNCTION `Rupture_Stock`(codigo
VARCHAR(50),
    cantidad INT
) RETURNS varchar(50) CHARSET utf8mb4 COLLATE utf8mb4_general_ci
BEGIN
       DECLARE StockRupture VARCHAR(50);
    DECLARE stockDisponible INT;

       SELECT quantityInStock INTO stockDisponible
       FROM products
    WHERE codigo = productCode;

    IF stockDisponible < cantidad THEN
    SET StockRupture = 'Ruptura de Stock';
    ELSE SET StockRupture = 'Stock Disponible';
    END IF;

RETURN StockRupture;
END
```

Explicación: Una vez pasados los parámetros, declaramos 2 variables dónde meteremos la cantidad de Stock y luego, en el IF, en caso de que la cantidad en stock es menor a la cantidad que pedimos, devolvemos “Stock disponible” y en el caso contrario, “Ruptura de Stock”

Función 5 - Ejercicio 11

Escribe una función denominada “Info1_Vendor” que devuelva el valor medio del precio de venta recomendado (MSRP) de los productos de un determinado proveedor que se recibirá como parámetro de entrada
Sentencia SQL:

```
SQL
CREATE DEFINER=`root`@`localhost` FUNCTION `Info1_Vendor`(codVendor
VARCHAR(50)
) RETURNS varchar(50) CHARSET utf8mb4 COLLATE utf8mb4_general_ci
BEGIN
       DECLARE mediaMSRP DECIMAL(10,2);

    SELECT AVG(MSRP) INTO mediaMSRP
       FROM products
    WHERE codVendor = productVendor;

RETURN mediaMSRP;
END
```

Explicación: Declaramos la media de MSRP donde haremos un AVG (La media) del MSRP y guardamos ahí el dato para luego retornarlo

Función 6 - Ejercicio 12

Escribe una función denominada “Customer_ofTheYear” que toma como parámetro el código de cliente y devuelve un mensaje que indica la fidelidad asociada al cliente dependiendo del valor del importe total de los pagos realizados: Pagos > 500000 ‘PLATINUM’; Pagos <= 500000 and >= 100000 ‘GOLD’; Pagos < 100000 and > 0 ‘SILVER
Sentencia SQL:

```
SQL
CREATE DEFINER=`root`@`localhost` FUNCTION `Customer_ofTheYear`(codCliente
INT
) RETURNS varchar(50) CHARSET utf8mb4 COLLATE utf8mb4_general_ci
BEGIN
       DECLARE importePagos DECIMAL(10,2);
    DECLARE mensaje VARCHAR(50);
```

```
    SELECT SUM(amount) INTO importePagos
    FROM payments
    WHERE codCliente = customerNumber;

    IF importePagos > 500000  THEN
    SET mensaje = 'PLATINUM';
    ELSEIF (importePagos <= 500000  AND importePagos >= 100000)  THEN
    SET mensaje = 'GOLD';
    ELSEIF (importePagos < 100000 AND importePagos > 0) THEN
    SET mensaje = 'SILVER';
    END IF;

RETURN mensaje;
END
```

Explicación: Declaramos la mensaje de importe Pagos donde guardamos el SUM de los payments y la otra que será el mensaje a retornar, y ya lo vamos condicionando con el IF para cambiar el mensaje a devolver

Cursor 1 - Ejercicio 13

Cursor — Código 1: analiza el procedimiento `ajustar_precios_stock()`. Debes insertar comentarios indicando: declaración de variables de apoyo, declaración del cursor, manejador de error (`NOT FOUND`), apertura del cursor, bucle de lectura y procesamiento de datos, y cierre del cursor. Después redacta una explicación funcional breve del procedimiento
Sentencia SQL:

```
SQL
-- CÓDIGO 1

DELIMITER //

CREATE PROCEDURE ajustar_precios_stock()
BEGIN
    -- 1. Declaración de variables
    DECLARE done INT DEFAULT FALSE;
    DECLARE p_code VARCHAR(15);
    DECLARE p_stock SMALLINT;
    DECLARE contador INT DEFAULT 0;

    -- 2. Cursor: productos con mucho stock
    DECLARE cur_productos CURSOR FOR
```

```
        SELECT productCode, quantityInStock
        FROM products
        WHERE quantityInStock > 5000;

    -- 3. Handler para fin de cursor
    DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = TRUE;

    -- 4. Abrir cursor
    OPEN cur_productos;

    -- 5. Bucle de lectura
    read_loop: LOOP
        FETCH cur_productos INTO p_code, p_stock;

        IF done THEN
            LEAVE read_loop;
        END IF;

        -- Lógica de actualización
        UPDATE products
        SET MSRP = MSRP * 0.90
        WHERE productCode = p_code;

        SET contador = contador + 1;
    END LOOP;

    -- 6. Cerrar cursor
    CLOSE cur_productos;

    -- Resultado final
    SELECT CONCAT('Proceso finalizado. Productos actualizados: ',
contador) AS Resultado;
END //

DELIMITER ;
```

Explicación:

Cursor 2 - Ejercicio 14

Cursor — Código 2: analiza el procedimiento `generar_directorio_oficina(IN codigo_oficina VARCHAR(10))`. Debes insertar comentarios indicando: declaración de variables de apoyo, declaración del cursor, manejador de error (`NOT FOUND`), apertura del

cursor, bucle de lectura y procesamiento de datos, y cierre del cursor. Después redacta una explicación funcional breve del procedimiento

Sentencia SQL:

```
SQL
DELIMITER //
CREATE PROCEDURE generar_directorio_oficina(IN codigo_oficina
VARCHAR(10))
BEGIN
 -- 1. DECLARAMOS VARIABLES
 DECLARE done INT DEFAULT FALSE;
 DECLARE v_nombre VARCHAR(50);
 DECLARE v_apellido VARCHAR(50);
 DECLARE v_email VARCHAR(100);

 -- 2. DCLARAMOS EL CURSOR JUNTO A LA CONSULTA QUE HARÁ
 DECLARE cur_empleados CURSOR FOR
 SELECT firstName, lastName, email
 FROM employees
 WHERE officeCode = codigo_oficina;
 -- 3. HANDLER PARA FIN DE CURSOS
 DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = TRUE;
 -- 4. ABRIR CURSOR
 OPEN cur_empleados;
 -- 5. Bucle de lectura
 bucle_empleados: LOOP
 FETCH cur_empleados INTO v_nombre, v_apellido, v_email;

 IF done THEN
 LEAVE bucle_empleados;
 END IF;
 -- Simulación de acción:
 SELECT CONCAT(v_apellido, ', ', v_nombre, ' - Contacto: ', v_email) AS
Ficha_Empleado;

 END LOOP;
 -- 6.Cerramos el cursor
 CLOSE cur_empleados;
END //
DELIMITER ;
```

Cursor 3 - Ejercicio 15

Cursor — Código 3: analiza la función `calcular_credito_pais(p_pais VARCHAR(50), p_umbral DECIMAL(10,2))`. Debes insertar comentarios indicando: declaración de variables de apoyo, declaración del cursor, manejador de error (`NOT FOUND`), apertura del cursor, bucle de lectura y procesamiento de datos, y cierre del cursor. Después redacta una explicación funcional breve del procedimiento/función

Sentencia SQL:

```SQL
DELIMITER //
CREATE FUNCTION calcular_credito_pais(p_pais VARCHAR(50), p_umbral
DECIMAL(10,2))
RETURNS DECIMAL(15,2)
BEGIN
 -- 1. DECLARAMOS VARIABLES
 DECLARE done INT DEFAULT FALSE;
 DECLARE v_credito_actual DECIMAL(10,2);
 DECLARE v_suma_total DECIMAL(15,2) DEFAULT 0;
 -- 2. DECLARAMOS EL CURSOS Y SU SELECT
 DECLARE cur_clientes CURSOR FOR
 SELECT creditLimit
 FROM customers
 WHERE country = p_pais;
 -- 3. DECLARAMOS EL HANDLER
 DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = TRUE;
 -- 4. ABRIMOS EL CURSOR
 OPEN cur_clientes;
 -- 5. ITERAMOS EL BUCLE DE LECTURA
 read_loop: LOOP
 FETCH cur_clientes INTO v_credito_actual;

 IF done THEN
 LEAVE read_loop;
 END IF;
 -- Solo ….
 IF v_credito_actual > p_umbral THEN
 SET v_suma_total = v_suma_total + v_credito_actual;
 END IF;
 END LOOP;
 -- 6. Cerramos el cursor
 CLOSE cur_clientes;

 RETURN v_suma_total;
END //
```

```
DELIMITER ;
```

Explicación:

Conclusiones personales

En esta práctica tanto mi compañero como yo hemos logrado dominar la creación de procedimientos y funciones de SQL , además de comprender la forma y lógica de los cursores. De tal manera que esta práctica en un futuro nos ayudará con las gestiones de automatización de procesos.